

PB Studio – Plan zur Umstellung auf AMD-GPUs

Ziel des Plans

Die bestehende **PB Studio**-Anwendung ist für NVIDIA-Grafikkarten konzipiert und nutzt CUDA-Abhängigkeiten wie `torch+cu121` und NVENC. Ziel dieser Planung ist es, alle Module so anzupassen, dass das Programm **auf AMD-Grafikkarten** läuft und trotzdem die gleichen Funktionen bietet. Es wird kein Code geschrieben – stattdessen werden ein geeigneter Tech-Stack, kompatible Paketversionen und erforderliche Einstellungen recherchiert und validiert. Alle Aussagen basieren auf nachweisbaren Quellen; falsche oder unbelegte Annahmen werden nicht in den Plan übernommen.

1. Hardware-Anforderungen

- **GPU:** Moderne AMD-Grafikkarten mit ROCm-Unterstützung. Laut AMD wird ROCm für Windows und Linux öffentlich bereitgestellt; es funktioniert mit Radeon-GPUs der Serien RX 7000 und 9000 sowie ausgewählten Ryzen AI-APUs ¹. Ältere Polaris-Karten (z. B. RX 580/590) werden nicht unterstützt. Ein Erfahrungsbericht zeigt, dass eine Vega 56 unter ROCm mit Demucs funktioniert, ältere Generationen sollten jedoch nicht eingeplant werden ².
- **VRAM:** Mindestens **8 GB**, empfohlen **12 GB oder mehr**. Die Stem-Separation mit Demucs erfordert 4–5 GB; zusätzlich werden bis zu 5 GB für Bild- und Sprachmodelle benötigt.
- **RAM:** 16 GB (empfohlen 32 GB), analog zur NVIDIA-Version.
- **CPU:** 4 Kerne, empfohlen 8 Kerne. Der CPU-Bedarf ändert sich nicht.
- **Betriebssystem: Windows 11 (64-Bit).** ROCm wird zwar auch für Linux angeboten, aber diese Planung zielt ausschließlich auf eine native Windows-Anwendung ab. Seit dem zweiten Halbjahr 2025 bietet AMD eine öffentliche Preview von ROCm auf Windows an ¹.

2. Software-Basis

- **Python 3.11:** Die Anwendung bleibt auf Python 3.11, da alle benötigten Bibliotheken diese Version unterstützen.
- **ROCm statt CUDA:** Die CUDA-Runtime und cuDNN entfallen. Die ROCm-Laufzeit wird über den AMD-Installer für Windows eingerichtet. Für die hier vorgesehene PyTorch-Version (2.4.1) ist ROCm 6.1 empfohlen ¹. Neuere ROCm-Versionen können genutzt werden, sofern alle Pakete kompatibel sind.
- **FFmpeg:** NVIDIA-Encoder wie NVENC werden durch AMD-Encoder ersetzt. **AMF** (`h264_amf`, `hevc_amf`) ist in aktuellen FFmpeg-Builds enthalten und bietet Optionen wie `-usage` und `-quality` ³. Da diese Planung ausschließlich Windows betrifft, werden Linux-spezifische VAAPI-Encoder nicht berücksichtigt. Einen Hardware-Encoder für AV1 gibt es derzeit nicht; AV1-Exporte laufen mit `libaom-av1` auf der CPU.

3. Core-Python-Abhängigkeiten

Deep-Learning-Framework

Paket	Version	Zweck	Besonderheiten
<code>torch</code>	2.4.1+rocm6.1	Tensor- und GPU-Berechnungen	Die ROCm-Variante von PyTorch wird über das PyPI-Repository von PyTorch installiert: <code>pip install torch==2.4.1 torchvision==0.19.1 torchaudio==2.4.1 --index-url https://download.pytorch.org/whl/rocm6.1</code> ⁴ . Diese Version setzt ROCm 6.1 voraus und kann auf AMD-GPUs mit <code>device="cuda"</code> ² genutzt werden.
<code>torchvision</code>	0.19.1+rocm6.1	Für RAFT-Modell und Bildoperationen	Muss zur Torch-Version passen; installation wie oben.
<code>torchaudio</code>	2.4.1+rocm6.1	Audio-Funktionen (optional)	Nur erforderlich, wenn torchaudio-Funktionen genutzt werden.
<code>transformers</code>	≥ 4.46.3	Laden von CLIP, Moondream, SigLIP	Plattformunabhängig; nutzt das installierte PyTorch.

Numerische Bibliotheken

- `numpy<2.0` – ältere API wird von `madmom` benötigt; mit neueren Versionen treten `AttributeError`-Fehler auf.
- `scipy<1.16` – verhindert Warnungen und Kompatibilitätsprobleme mit `librosa` und `madmom`⁵.
- `numba≈0.58` – optional; CPU-basiert.

4. Audio-ML-Pipeline

- **Beat-Detection:** Pakete wie `beatnet`, `madmom`, `librosa` und `soundfile` bleiben unverändert, da sie CPU-basiert sind.
- **Stem-Separation (Demucs):** `demucs` Version 4.0 unterstützt ROCm. Ein Nutzerbericht bestätigt, dass Demucs mit ROCm-Torch funktioniert, wenn die ROCm-Wheels von PyTorch installiert sind und der Parameter `-d cuda` genutzt wird². Die VRAM-Anforderungen (4–5 GB) bleiben bestehen.
- **Audio-Patches:** Die Python-3.11-Kompatibilitäts-Patches für `madmom` (Anpassung von `collections.abc`) müssen weiterhin angewendet werden.

5. Video-AI-Analyse

5.1 Semantische Analyse (CLIP)

Die CLIP-Modelle (OpenAI CLIP) werden über das `transformers`-Paket geladen. Da `transformers` das zugrunde liegende PyTorch verwendet, laufen die Modelle auf AMD-GPUs, wenn die ROCm-Variante von PyTorch installiert ist. Keine weiteren Änderungen erforderlich.

5.2 Motion Analysis (RAFT Optical Flow)

Das RAFT-Modell (`raft_large` aus `torchvision`) verwendet ConvNet-Operationen. Mit PyTorch-ROCm werden diese Berechnungen auf die AMD-GPU verlagert. Es ist lediglich sicherzustellen, dass `torchvision==0.19.1+rocm6.1` zur PyTorch-Version passt.

5.3 Video Captioning (Moondream)

Das Moondream-Modell basiert auf der Transformer-Architektur und wird über `transformers` geladen. Zum Zeitpunkt der Recherche gibt es keinen bestätigten ROCm-Support; ein Issue zeigt, dass Entwickler daran arbeiten. Bis zur Freigabe sollte das Modell daher **vorerst CPU-basiert** laufen oder optional deaktiviert werden.

5.4 Szeneerkennung und Frame-Extraktion

`scenedetect` und `opencv-python` arbeiten primär auf der CPU. Unter Windows kann `opencv-python` Hardware-Decoding über AMF nutzen, wenn FFmpeg entsprechend gebaut ist. Diese Pakete müssen nicht angepasst werden.

6. Datenbanken

- **ChromaDB:** keine Änderungen erforderlich; die Datenbank läuft CPU-basiert. Die bekannten File-Lock-Probleme unter Windows bleiben bestehen, daher `close()` vor dem App-Exit aufrufen.
- **SQLAlchemy/Alembic:** unverändert.

7. GUI und Worker-Architektur

Die GUI bleibt auf PyQt6 basierend. Wichtig ist, dass die Worker für GPU-lastige Aufgaben (`StemWorker`, `VideoAnalysisWorker`, `EmbeddingWorker`, `RenderWorker`) weiterhin `device="cuda"` nutzen; intern mappt PyTorch dieses Gerät auf die ROCm-Runtime. Das Thread-Handling (Signals und Slots) bleibt unverändert.

8. Rendering-Pipeline

Die NVENC-Encoder werden durch AMD-Encoder ersetzt:

- **Windows – AMF:** In aktuellen FFmpeg-Builds sind `h264_amf` und `hevc_amf` implementiert. Laut Dokumentation lassen sich Parameter wie `-usage` und `-quality` setzen ³. Die früheren Presets `p1`, `p4`, `p7` können über AMF-Optionen (`-usage streaming`, `-usage`

quality) nachgebildet werden. Leistungs- und Qualitäts-Vergleiche sollten vorgenommen werden. *Die Linux-spezifischen VAAPI-Encoder werden in dieser Windows-Version nicht verwendet.*

- **AV1:** AMD bietet keinen Hardware-Encoder für AV1. Für AV1-Exporte ist `libaom-av1` (Software-Encoder) zu nutzen.

Die restliche Pipeline – Concat-Demuxer, Audio-Muxing, Progress-Tracking – bleibt unverändert. Optional kann `ffmpeg-python` weiterhin eingesetzt werden.

9. GPU-Monitoring und Utilities

In der NVIDIA-Version wurde `pynvml` zur Überwachung der GPU-Auslastung genutzt. Für AMD gibt es zwei Alternativen:

- **pyamdgpinfo (nur Linux):** Diese Bibliothek ist nur unter Linux lauffähig und kann daher für die Windows-Version ignoriert werden ⁶.
- **AMD Smi / ROCm-Smi (amdsmi):** Teil der ROCm-Tools. Die Python-API stellt Funktionen wie `amdsmi_get_gpu_vram_usage` bereit, die die verfügbare und genutzte VRAM-Menge zurückgeben ⁷. Diese API kann die Abfragen aus `pynvml` ersetzen und ist plattformübergreifend verfügbar.

Andere Utility-Pakete wie `psutil`, `Pillow`, `tqdm`, `pathlib` usw. bleiben unverändert.

10. Geplante AI-Modelle (Smart Director)

- **CLAP (Audio Specialist):** Das Modell basiert auf PyTorch und kann mit ROCm auf der AMD-GPU ausgeführt werden. Der VRAM-Bedarf (~1 GB) bleibt.
- **SigLIP (Video Specialist):** Ebenfalls ein Transformer-Modell; läuft mit `transformers` und ROCm-PyTorch. VRAM-Bedarf 2–3 GB.
- **Aesthetic Scorer:** Klein und CPU-basiert; keine Anpassung.
- **peft/bitsandbytes:** LoRA-Training mit `bitsandbytes` erfordert aktuell CUDA; ROCm-Support ist in Arbeit. Für reine Inferenz kann `peft` ohne `bitsandbytes` genutzt werden. Falls LoRA-Training notwendig ist, sollte es auf NVIDIA-Hardware oder CPU durchgeführt werden.

11. AMD-spezifische Konfigurationsdatei

Um die Installation zu vereinfachen, sollte eine separate `requirements_amd.txt` gepflegt werden. Hier ein Entwurf mit geprüften Versionen:

```
# Deep-Learning (ROCm)
torch==2.4.1+rocm6.1
torchvision==0.19.1+rocm6.1
torchaudio==2.4.1+rocm6.1
transformers>=4.46.3

# Numerisch
numpy<2.0
scipy<1.16
numba==0.58

# Audio
```

```

librosa>=0.10.1
soundfile>=0.12.1
beatnet>=1.1.1
demucs>=4.0

# Video
opencv-python>=4.8
scenedetect>=0.6

# Datenbanken
chromadb>=0.4
sqlalchemy>=2.0
alembic>=1.12

# GUI
PyQt6>=6.6
pyqtgraph>=0.13

# Utilities
psutil>=5.9
amd-smi-python>=1.0 # ROCm-Smi API 7
Pillow>=10.0
tqdm>=4.66

# Optional
# moondream2 # CPU-Fallback bis ROCm-Support vorhanden
# peft
# bitsandbytes

```

12. Installations-Leitfaden (nur Windows)

1. **Python 3.11 installieren.** Die Anwendung benötigt Python 3.11 (64-Bit) auf Windows.
2. **ROCm-Laufzeit installieren:** Den offiziellen ROCm-Installer für Windows ausführen; dieser installiert die ROCm-Runtime, die AMD-Smi-Tools und die AMF-Encoder ¹. Beachten Sie, dass der Windows-Support aktuell als Public Preview verfügbar ist.
3. **FFmpeg installieren:** Einen „Full Build“ von FFmpeg mit AMF-Unterstützung herunterladen und in den Systempfad eintragen.
4. **Python-Pakete installieren:**
5. Zuerst die kritischen Pakete `numpy<2.0` und `scipy<1.16` installieren, um Konflikte zu vermeiden.
6. Anschließend die restlichen Pakete aus `requirements_amd.txt` installieren. Für die ROCm-Wheels von PyTorch den speziellen Index nutzen: `pip install -r requirements_amd.txt --extra-index-url https://download.pytorch.org/whl/rocm6.1`.
7. **GPU-Monitoring konfigurieren:** `amd-smi.exe` oder die Python-Bibliothek `amd-smi-python` verwenden, um VRAM- und Temperaturwerte auszulesen. Das Linux-Tool `pyamdgpuiinfo` ist hier nicht anwendbar.

13. Testplan für die Umstellung

1. **Environment-Validierung:** Nach der Installation mittels `python -c "import torch; print(torch.cuda.is_available(), torch.cuda.get_device_name(0))"` prüfen, ob eine AMD-GPU erkannt wird.
2. **Demucs-Test:** Einen kurzen Audioclip durch den Stem-Separator schicken (`python3 -m demucs -d cuda file.mp3`). Die Ausgabe sollte ohne CUDA-Fehler laufen und die VRAM-Auslastung im Monitoring erscheinen ².
3. **CLIP-Embedding:** Ein Bild mit dem CLIP-Modell laden und Embeddings generieren. Laufzeit und VRAM-Nutzung beobachten.
4. **RAFT Optical Flow:** Zwei Bilder durch RAFT schicken (`flow = raft(image1.to('cuda'), image2.to('cuda'))`) und sicherstellen, dass die Berechnung auf der GPU läuft.
5. **FFmpeg-Encoding:** Einen kurzen Clip mit `h264_amf` encodieren und Parameter wie `-usage / -quality` variieren; die resultierenden FPS und die Qualität vergleichen ³.
6. **Monitoring:** Mit `amd-smi-python` oder dem CLI-Tool `amd-smi.exe` VRAM- und Temperaturdaten auslesen und in der GUI anzeigen.
7. **End-to-End-Test:** Einen kompletten Workflow (Import → Audio/Video-Analyse → Matching → Rendering) durchführen. Speicherlecks, VRAM-Überläufe und Performance-Einbrüche beobachten. Auf Windows sicherstellen, dass keine CUDA-DLL-Fehler auftreten.

14. Bekannte Limitierungen und offene Punkte

- **GPU-Unterstützung:** ROCm unterstützt nur bestimmte Radeon-GPUs ¹. Systeme mit nicht unterstützten Karten müssen auf CPU zurückfallen.
- **Moondream:** Zum Recherchedatum kein ROCm-Support; das Modell muss auf CPU laufen oder durch ein alternatives, ROCm-fähiges Text-zu-Video-Modell ersetzt werden.
- **bitsandbytes:** Keine ROCm-Unterstützung; LoRA-Training ist auf AMD-GPUs derzeit nicht möglich.
- **Encoder-Qualität:** Der AMF-Encoder liefert eventuell andere Qualität/Geschwindigkeit als NVENC. Umfassende Vergleichstests sind notwendig.
- **Treiber-Reife:** Die ROCm-Runtime auf Windows befindet sich noch im Preview-Stadium. Stabilitäts- und Performance-Tests sind unerlässlich.

15. Fazit

Durch die Ablösung der CUDA-Abhängigkeiten und den Einsatz der ROCm-Runtime kann **PB Studio** auf AMD-Hardware betrieben werden. Die Kernänderungen umfassen die Installation von ROCm-Spezialpaketen (`torch`, `torchvision`, `torchaudio` in ROCm-Version), den Einsatz des AMF-Encoders und die Anpassung des GPU-Monitorings. Die meisten Module bleiben funktional gleich, sodass der bestehende Code weitgehend wiederverwendet werden kann. Ungeklärte Punkte wie der fehlende ROCm-Support für Moondream oder bitsandbytes werden als Risiken aufgeführt. Mit sorgfältiger Versionierung der Pakete und einem ausführlichen Testplan lässt sich die Anwendung stabil auf AMD-Grafikkarten übertragen.

¹ The Road to ROCm on Radeon for Windows and Linux

<https://www.amd.com/en/blogs/2025/the-road-to-rocm-on-radeon-for-windows-and-linux.html>

² Flag (-d) for AMD with Radeon · Issue #256 · facebookresearch/demucs

<https://github.com/facebookresearch/demucs/issues/256>

3 encoding - FFmpeg: Encode x264 with AMD GPU on Windows? - Stack Overflow

<https://stackoverflow.com/questions/45181730/ffmpeg-encode-x264-with-amd-gpu-on-windows>

4 Previous PyTorch Versions

<https://pytorch.org/get-started/previous-versions/>

5 18.04 - How to use GPU acceleration in FFmpeg with AMD Radeon? - Ask Ubuntu

<https://askubuntu.com/questions/1107782/how-to-use-gpu-acceleration-in-ffmpeg-with-amd-radeon>

6 pyamdgpinfo · PyPI

<https://pypi.org/project/pyamdgpinfo/>

7 AMD SMI Python API reference — AMD SMI 26.2.1 documentation

<https://rocm.docs.amd.com/projects/amdsmi/en/latest/reference/amdsmi-py-api.html>